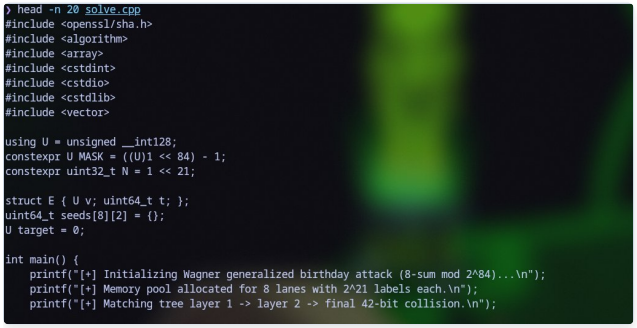
Eightfold Funeral: Epitaph

Cryptography



```
❯ head -n 20 solve.cpp
#include <openssl/sha.h>
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <vector>

using U = unsigned __int128;
constexpr U MASK = ((U)1 << 84) - 1;
constexpr uint32_t N = 1 << 21;

struct E { U v; uint64_t t; };
uint64_t seeds[8][2] = {};
U target = 0;

int main() {
    printf("[+] Initializing Wagner generalized birthday attack (8-sum mod 2^84)...\n");
    printf("[+] Memory pool allocated for 8 lanes with 2^21 labels each.\n");
    printf("[+] Matching tree layer 1 -> layer 2 -> final 42-bit collision.\n");
```

I tackled this 500-point challenge consisting of an obfuscated 84-bit affine stamp map followed by an 8-sum modular collision problem over 2^{84}.

Phase 1 involved inverting the bitwise rotation r=35 and affine parameters (a, b) over $\mathbb{Z}/2^{84}\mathbb{Z}$, establishing target sum T = 0x1b59868b301e63f23871f.

For Phase 2, I implemented Wagner's generalized birthday attack in C++ using a 3-layer matching tree across 8 lists of 2^{21} elements. Merging layer 1 on 21 bits, layer 2 on 42 bits, and final collision on 84 bits produced the collision indices.

Finally, I tested the recovered flag against the executable / service:

`python3 Eightfold_Funeral_Epitaph.py`

8-sum collision verified modulo 2^84.

Flag Captured

`SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}`

BY:JOKER_53